1. PERUMUSAN MASALAH DAN DATASET

Tujuan Analisis : Proses persetujuan pinjaman merupakan keputusan penting bagi lembaga keuangan karena berkaitan dengan risiko kredit.

SUMBER DATA

Loan Approval Prediction Dataset

Data ini berasal dari kaggle

```
import pandas as pd
import numpy as np

df = pd.read_excel('loan_approval_dataset.xlsx')

df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 4269,\n  \"fields\":
[\n    {\n      \"column\": \"loan_id\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 1232,\n        \"min\": 1,\n
\"max\": 4269,\n        \"num_unique_values\": 4269,\n
\"samples\": [\n          1704,\n          1174,\n          309\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \" no_of_dependents\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1,\n        \"min\": 0,\n        \"max\": 5,\n
\"num_unique_values\": 6,\n        \"samples\": [\n          2,\n
0,\n          1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"
education\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          \" Not Graduate\",\n          \" Graduate\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \" self_employed\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          \"
Yes\",\n          \" No\"\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \" income_annum\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 2806839,\n        \"min\":
200000,\n        \"max\": 9900000,\n        \"num_unique_values\":
98,\n        \"samples\": [\n          6200000,\n          9300000\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \" loan_amount\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
9043362,\n        \"min\": 300000,\n        \"max\": 39500000,\n
\"num_unique_values\": 378,\n        \"samples\": [\n
```

```
25800000,\n          26100000\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \" loan_term\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 5,\n        \"min\": 2,\n
\"max\": 20,\n        \"num_unique_values\": 10,\n        \"samples\":
[\n          14,\n          8\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \" cibil_score\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 172,\n        \"min\": 300,\n
\"max\": 900,\n        \"num_unique_values\": 601,\n
\"samples\": [\n          859,\n          414\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \" residential_assets_value\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
6503636,\n        \"min\": -100000,\n        \"max\": 29100000,\n
\"num_unique_values\": 278,\n        \"samples\": [\n
700000,\n          3500000\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \" commercial_assets_value\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 4388966,\n        \"min\":
0,\n        \"max\": 19400000,\n        \"num_unique_values\": 188,\n
\"samples\": [\n          13500000,\n          14600000\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \" luxury_assets_value\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
9103753,\n        \"min\": 300000,\n        \"max\": 39200000,\n
\"num_unique_values\": 379,\n        \"samples\": [\n
15300000,\n          12100000\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \" bank_asset_value\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 3250185,\n        \"min\":
0,\n        \"max\": 14700000,\n        \"num_unique_values\": 146,\n
\"samples\": [\n          4800000,\n          14400000\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \" loan_status\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          \"
Rejected\",\n          \" Approved\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"df"}
```


```
df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 4269,\n  \"fields\":
[\n    {\n      \"column\": \"loan_id\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 1232,\n        \"min\": 1,\n
\"max\": 4269,\n        \"num_unique_values\": 4269,\n
\"samples\": [\n          1704,\n          1174,\n          309\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \" no_of_dependents\",\n
```

```
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1,\n        \"min\": 0,\n        \"max\": 5,\n
\"num_unique_values\": 6,\n        \"samples\": [\n          2,\n
0,\n          1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"
education\",\n      \"properties\": {\n        \"dtype\":
\"category\",\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          \" Not Graduate\",\n          \" Graduate\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \" self_employed\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          \"
Yes\",\n          \" No\"\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \" income_annum\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 2806839,\n        \"min\":
200000,\n        \"max\": 9900000,\n        \"num_unique_values\":
98,\n        \"samples\": [\n          6200000,\n          9300000\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \" loan_amount\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
9043362,\n        \"min\": 300000,\n        \"max\": 39500000,\n
\"num_unique_values\": 378,\n        \"samples\": [\n
25800000,\n          26100000\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \" loan_term\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 5,\n        \"min\": 2,\n
\"max\": 20,\n        \"num_unique_values\": 10,\n        \"samples\":
[\n          14,\n          8\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \" cibil_score\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 172,\n        \"min\": 300,\n
\"max\": 900,\n        \"num_unique_values\": 601,\n
\"samples\": [\n          859,\n          414\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \" residential_assets_value\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
6503636,\n        \"min\": -100000,\n        \"max\": 29100000,\n
\"num_unique_values\": 278,\n        \"samples\": [\n
700000,\n          3500000\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \" commercial_assets_value\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 4388966,\n        \"min\":
0,\n        \"max\": 19400000,\n        \"num_unique_values\": 188,\n
\"samples\": [\n          13500000,\n          14600000\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \" luxury_assets_value\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
9103753,\n        \"min\": 300000,\n        \"max\": 39200000,\n
```

\"num_unique_values\": 379,\n   \"samples\": [\n
15300000,\n   12100000\n   ],\n   \"semantic_type\":
\"\",\n   \"description\": \"\"\n   }\n   },\n   {\n
\"column\": \" bank_asset_value\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 3250185,\n   \"min\":
0,\n   \"max\": 14700000,\n   \"num_unique_values\": 146,\n
\"samples\": [\n   4800000,\n   14400000\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \" loan_status\",\n
\"properties\": {\n   \"dtype\": \"category\",\n
\"num_unique_values\": 2,\n   \"samples\": [\n   \"
Rejected\",\n   \" Approved\"\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe","variable_name":"df"}

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4269 entries, 0 to 4268
Data columns (total 13 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   loan_id                   4269 non-null   int64
 1    no_of_dependents         4269 non-null   int64
 2    education                4269 non-null   object
 3    self_employed            4269 non-null   object
 4    income_annum             4269 non-null   int64
 5    loan_amount              4269 non-null   int64
 6    loan_term                4269 non-null   int64
 7    cibil_score              4269 non-null   int64
 8    residential_assets_value 4269 non-null   int64
 9    commercial_assets_value  4269 non-null   int64
 10   luxury_assets_value      4269 non-null   int64
 11   bank_asset_value         4269 non-null   int64
 12   loan_status              4269 non-null   object
dtypes: int64(10), object(3)
memory usage: 433.7+ KB
```

```
df = df.drop(columns=['loan_id'])
```

2. EDA

a. Membuat Statistik Deskriptif

```
df.describe()
```

{"summary":"{\n \"name\": \"df\",\n \"rows\": 8,\n \"fields\": [\n
{\n   \"column\": \" no_of_dependents\",\n   \"properties\": {\n

```
\"dtype\": \"number\",\n        \"std\": 1508.4518117060454,\n
\"min\": 0.0,\n        \"max\": 4269.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
2.4987116420707425,\n          3.0,\n          4269.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \" income_annum\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
3437390.0912845903,\n        \"min\": 4269.0,\n        \"max\":
9900000.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\
n          5059123.9166081045,\n          5100000.0,\n
4269.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"
loan_amount\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 12837065.839384567,\n        \"min\":
4269.0,\n        \"max\": 39500000.0,\n        \"num_unique_values\":
8,\n        \"samples\": [\n          15133450.456781447,\n
14500000.0,\n          4269.0\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \" loan_term\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 1505.7642551387924,\n
\"min\": 2.0,\n        \"max\": 4269.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
10.900445069102835,\n          10.0,\n          4269.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \" cibil_score\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1339.1773722653782,\n        \"min\": 172.43040073575904,\n
\"max\": 4269.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          599.9360505973295,\n          600.0,\n
4269.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"
residential_assets_value\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 9464960.95825069,\n
\"min\": -100000.0,\n        \"max\": 29100000.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
7472616.537830873,\n          5600000.0,\n          4269.0\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"
commercial_assets_value\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 6320013.940564741,\n
\"min\": 0.0,\n        \"max\": 19400000.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
4973155.3056922,\n          3700000.0,\n          4269.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \" luxury_assets_value\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
12779780.356898261,\n        \"min\": 4269.0,\n        \"max\":
39200000.0,\n        \"num_unique_values\": 8,\n        \"samples\":
[\n          15126305.926446475,\n          14600000.0,\n
```

```
4269.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"
bank_asset_value\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 4747818.903599107,\n        \"min\":
0.0,\n        \"max\": 14700000.0,\n        \"num_unique_values\": 8,\
n        \"samples\": [\n          4976692.433825252,\n
4600000.0,\n          4269.0\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe"}
```


```
df.describe(include='object')
```

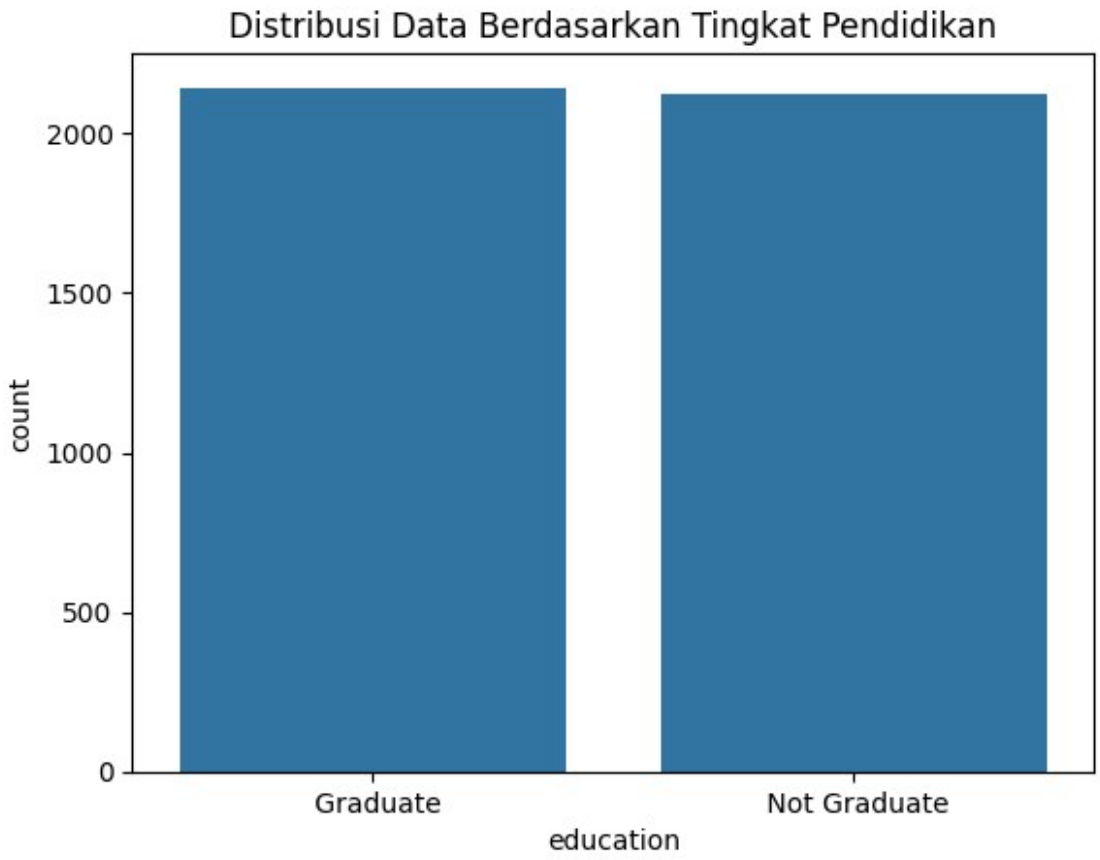

```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 4,\n  \"fields\": [\n
{\n      \"column\": \" education\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 4,\n
\"samples\": [\n          2,\n          \"2144\",\n          \"4269\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"
self_employed\",\n      \"properties\": {\n        \"dtype\":
\"string\",\n        \"num_unique_values\": 4,\n        \"samples\":
[\n          2,\n          \"2150\",\n          \"4269\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \" loan_status\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n          2,\n
\"2656\",\n          \"4269\"\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe"}
```


b. Memvisualisasikan Data

Bar Chart

```
df.columns = df.columns.str.strip()

#Menampilkan nilai unik dalam kolom education
unique_values = df['education'].unique()
print(unique_values)

#Analisis Distribusi Kategori
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x='education', data=df)
plt.title('Distribusi Data Berdasarkan Tingkat Pendidikan')

plt.show()
```

```
[' Graduate' ' Not Graduate']
```

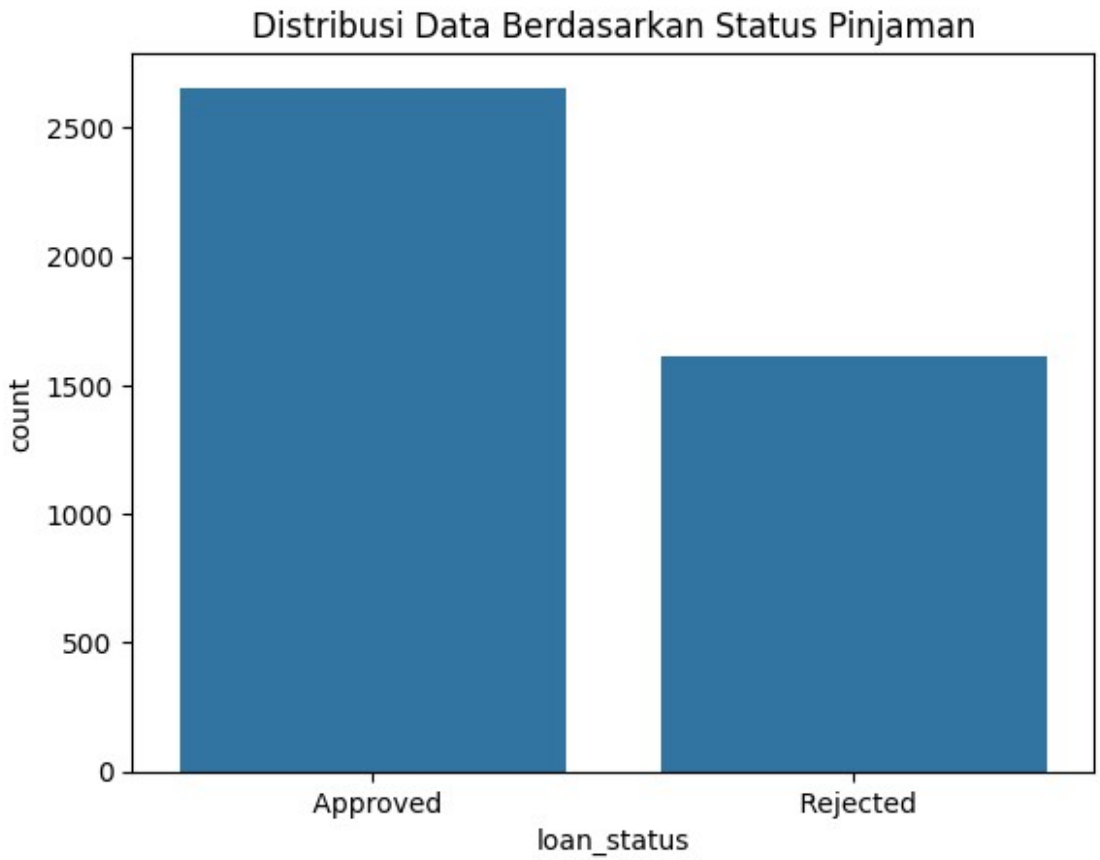


INTERPRETASI

Dari barchart terlihat jumlah pemohon dengan pendidikan Graduate sedikit lebih banyak dibandingkan Not Graduate. Namun perbedaannya tidak terlalu jauh sehingga distribusi data berdasarkan tingkat pendidikan dapat dianggap seimbang.

```
# Menampilkan nilai unik dalam kolom loan_status
unique_values = df['loan_status'].unique()
print(unique_values)

# Analisis Distribusi Kategori
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x='loan_status', data=df)
plt.title('Distribusi Data Berdasarkan Status Pinjaman')

plt.show()
```

```
[' Approved' ' Rejected']
```

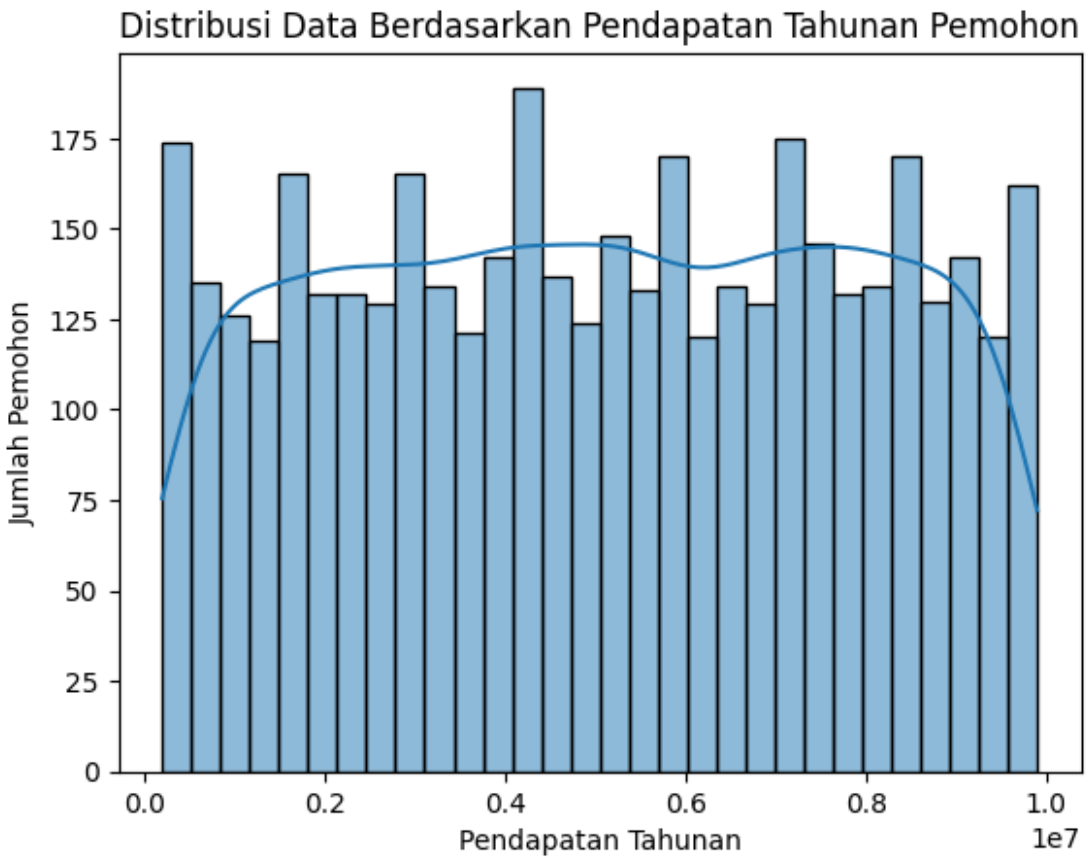


INTERPRETASI

Dari barchart terlihat jumlah data dengan status pinjaman Approved lebih banyak dibandingkan Rejected. Perbedaannya cukup jauh sehingga distribusi data berdasarkan status pinjaman tidak seimbang.

Histogram

```
# Histogram berdasarkan Pendapatan Tahunan
sns.histplot(data=df, x='income_annum', bins=30, kde=True)
plt.title('Distribusi Data Berdasarkan Pendapatan Tahunan Pemohon')
plt.xlabel('Pendapatan Tahunan')
plt.ylabel('Jumlah Pemohon')
plt.show()
```



INTERPRETASI

Berdasarkan histogram, terlihat bahwa pendapatan tahunan pemohon tersebar pada berbagai rentang pendapatan dengan jumlah yang relatif merata. Tidak terdapat satu rentang pendapatan tertentu yang sangat mendominasi dibandingkan rentang lainnya.

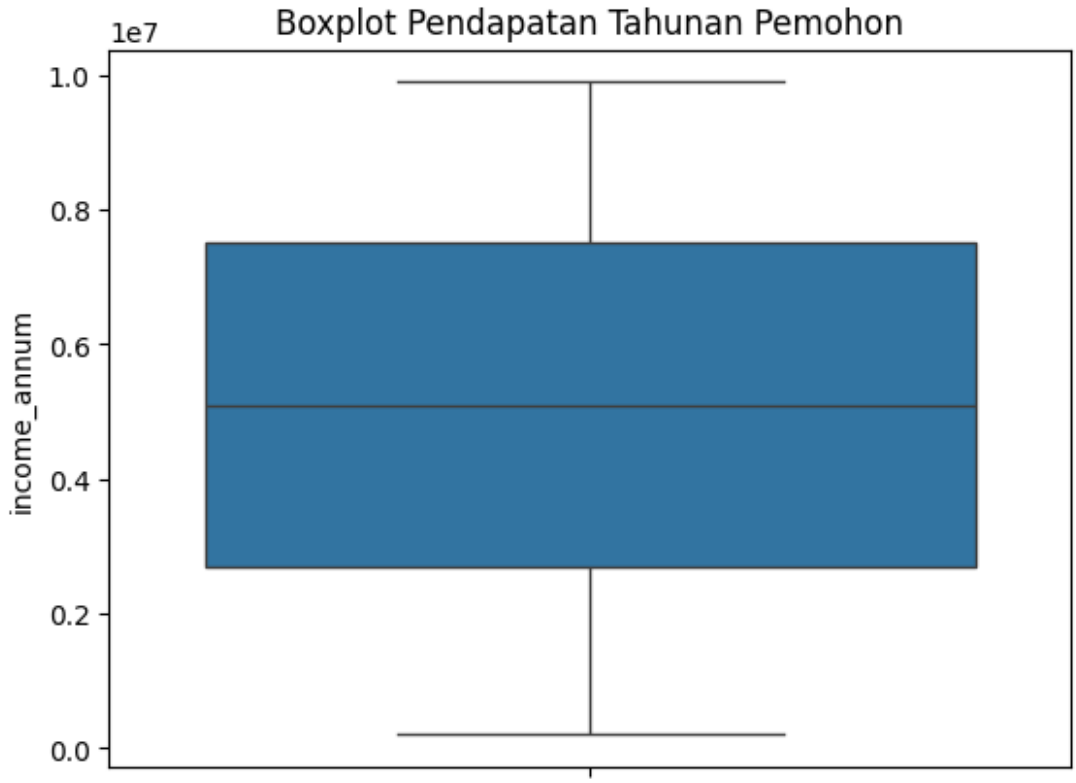
Pola distribusi ini menunjukkan bahwa data pendapatan tahunan tidak mengikuti distribusi normal yang jelas. Sebaran data cukup luas dan mencakup pemohon dengan pendapatan rendah hingga tinggi, sehingga dataset mampu merepresentasikan berbagai kondisi ekonomi pemohon pinjaman.

Dalam konteks analisis persetujuan pinjaman, variabel income_annum penting untuk diperhatikan karena pendapatan tahunan dapat memengaruhi kemampuan pemohon dalam memenuhi kewajiban pembayaran pinjaman. Pemohon dengan pendapatan yang lebih tinggi cenderung memiliki kemampuan finansial yang lebih baik sehingga berpotensi lebih mudah memperoleh persetujuan pinjaman dibandingkan pemohon dengan pendapatan yang lebih rendah.

Boxplot

```
# Boxplot Pendapatan Tahunan

sns.boxplot(y='income_annum', data=df)
plt.title('Boxplot Pendapatan Tahunan Pemohon')
```

```
plt.show()
```



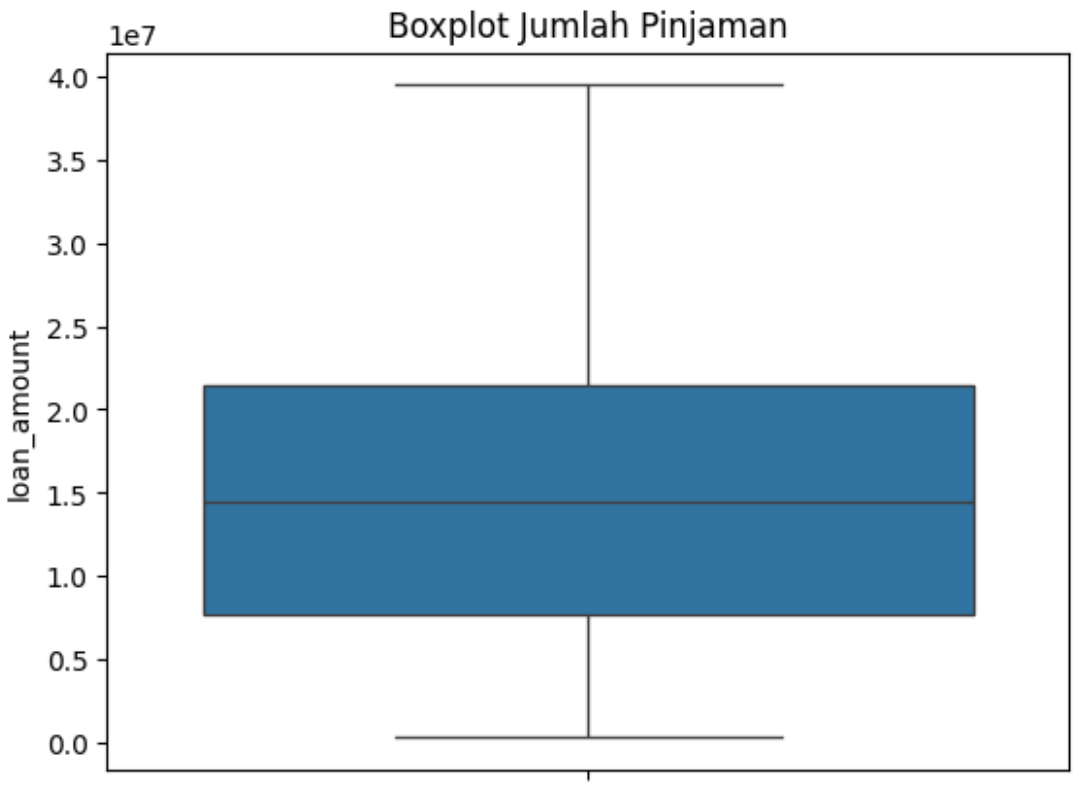
INTERPRETASI

Berdasarkan boxplot, terlihat bahwa data pendapatan tahunan pemohon memiliki sebaran yang cukup luas, mulai dari pendapatan rendah hingga pendapatan tinggi. Nilai median berada di sekitar bagian tengah boxplot yang menunjukkan bahwa data relatif tersebar secara merata.

Selain itu, tidak terlihat adanya titik data yang berada di luar whisker sehingga tidak ditemukan outlier yang signifikan pada variabel income_annum. Hal ini menunjukkan bahwa sebagian besar data masih berada dalam rentang yang wajar dan tidak terdapat nilai ekstrem yang dapat memengaruhi analisis.

Dalam konteks analisis persetujuan pinjaman, variabel income_annum penting untuk diperhatikan karena pendapatan tahunan mencerminkan kemampuan finansial pemohon. Semakin tinggi pendapatan yang dimiliki, semakin besar kemungkinan pemohon memiliki kemampuan untuk memenuhi kewajiban pembayaran pinjaman sehingga berpotensi memengaruhi keputusan persetujuan pinjaman.

```
# Boxplot Jumlah Pinjaman

sns.boxplot(data=df, y='loan_amount')
plt.title('Boxplot Jumlah Pinjaman')
plt.show()
```



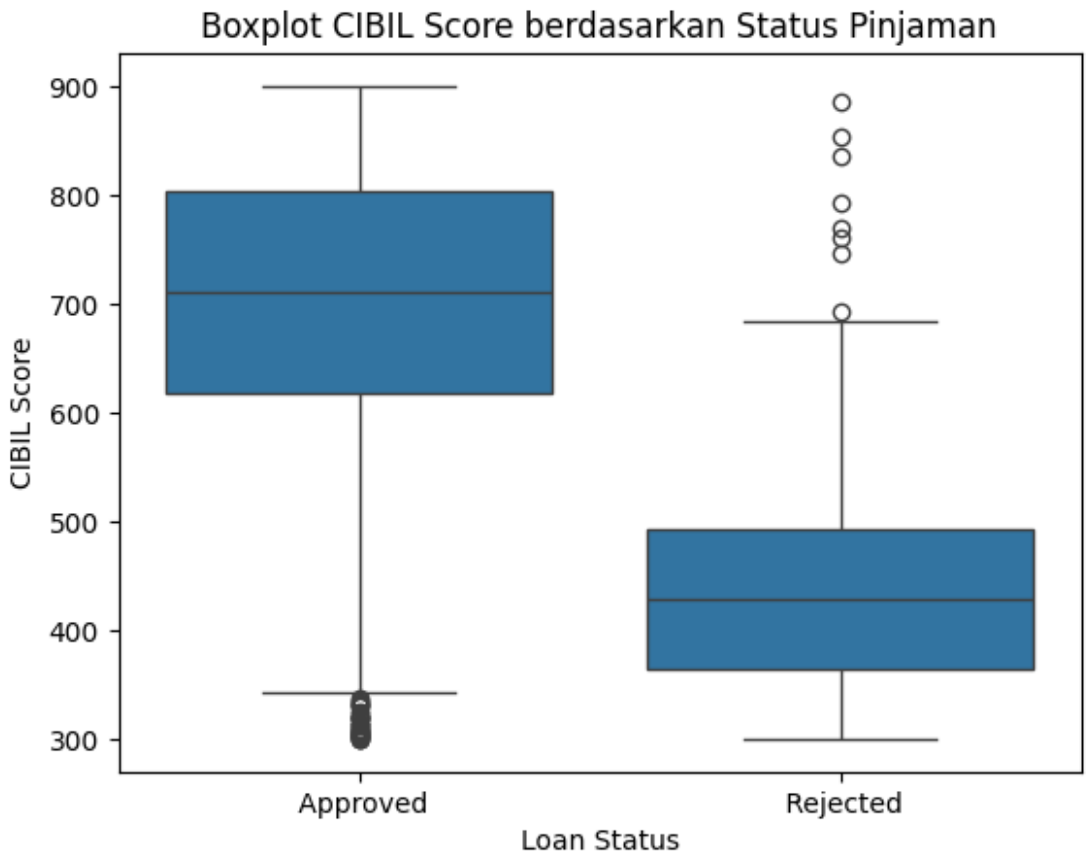
INTERPRETASI

Berdasarkan boxplot, terlihat bahwa jumlah pinjaman yang diajukan pemohon memiliki rentang yang cukup luas, mulai dari nilai pinjaman yang rendah hingga tinggi. Nilai median berada di sekitar bagian tengah boxplot yang menunjukkan bahwa sebagian besar data tersebar di sekitar nilai tersebut.

Selain itu, tidak terlihat adanya titik data yang berada di luar whisker sehingga tidak ditemukan outlier yang signifikan pada variabel loan_amount. Hal ini menunjukkan bahwa sebagian besar nilai pinjaman masih berada dalam rentang yang wajar dan tidak terdapat nilai ekstrem yang dapat memengaruhi analisis.

Dalam konteks analisis persetujuan pinjaman, variabel loan_amount penting untuk diperhatikan karena besarnya pinjaman yang diajukan dapat memengaruhi keputusan persetujuan pinjaman. Semakin besar jumlah pinjaman yang diajukan, semakin besar pula risiko yang harus dipertimbangkan oleh lembaga keuangan dalam memberikan persetujuan pinjaman.

```
# Boxplot CIBIL Score berdasarkan Status Pinjaman

sns.boxplot(data=df, x='loan_status', y='cibil_score')
plt.title('Boxplot CIBIL Score berdasarkan Status Pinjaman')
plt.xlabel('Loan Status')
plt.ylabel('CIBIL Score')

plt.show()
```



INTERPRETASI

Berdasarkan boxplot, terlihat bahwa pemohon dengan status pinjaman Approved memiliki CIBIL Score yang cenderung lebih tinggi dibandingkan pemohon dengan status Rejected. Nilai median CIBIL Score pada kelompok Approved berada di atas 700, sedangkan pada kelompok Rejected berada di sekitar 400.

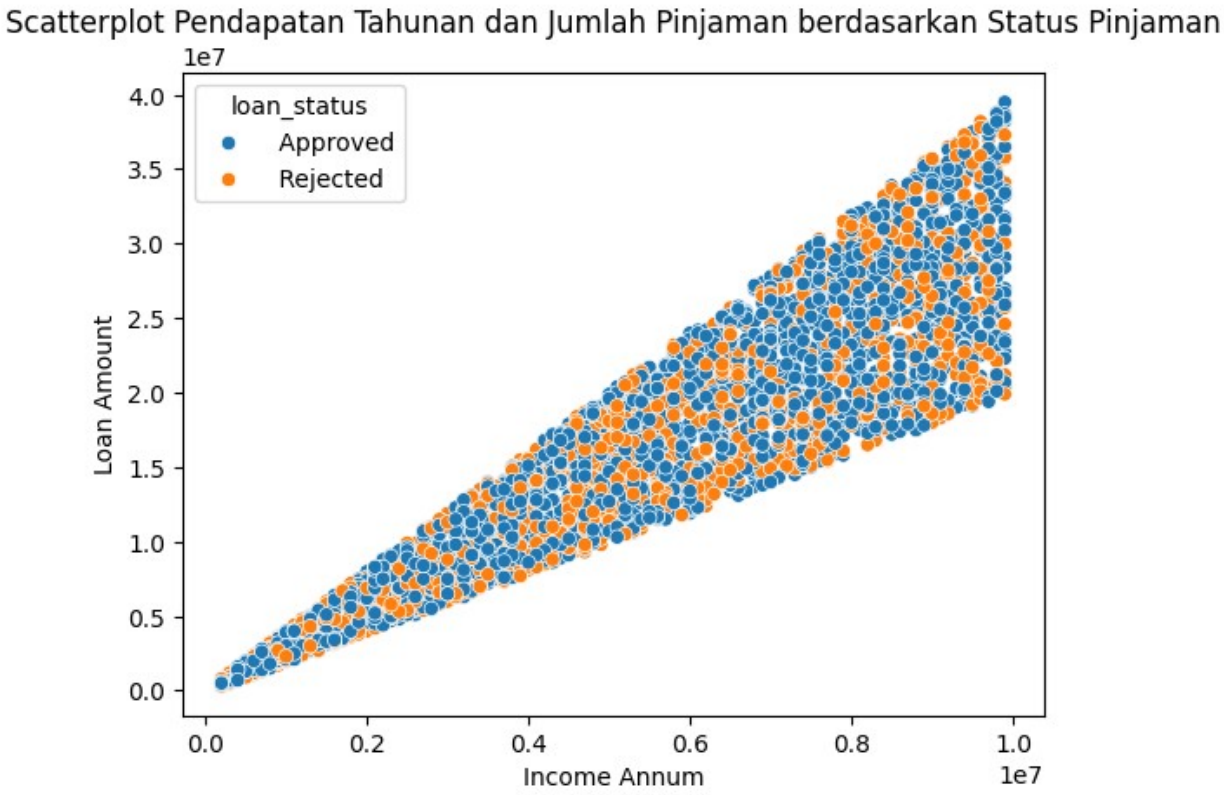
Selain itu, terdapat perbedaan distribusi yang cukup jelas antara kedua kelompok. Sebagian besar pemohon yang mendapatkan persetujuan pinjaman memiliki skor kredit yang tinggi, sedangkan pemohon yang ditolak cenderung memiliki skor kredit yang lebih rendah. Meskipun terdapat beberapa outlier pada kedua kelompok, pola perbedaan antara Approved dan Rejected tetap terlihat dengan jelas.

Dalam konteks analisis persetujuan pinjaman, variabel CIBIL Score sangat penting karena mencerminkan riwayat dan kelayakan kredit pemohon. Semakin tinggi skor kredit yang dimiliki, semakin besar kemungkinan pemohon memperoleh persetujuan pinjaman. Oleh karena itu, CIBIL Score dapat menjadi salah satu variabel yang berpengaruh dalam proses klasifikasi status pinjaman.

Scatter Plot

```
# Scatterplot Income Annum vs Loan Amount berdasarkan Loan Status

sns.scatterplot(
```

```
    data=df,
    x='income_annum',
    y='loan_amount',
    hue='loan_status'
)

plt.title('Scatterplot Pendapatan Tahunan dan Jumlah Pinjaman
berdasarkan Status Pinjaman')
plt.xlabel('Income Annum')
plt.ylabel('Loan Amount')

plt.show()
```



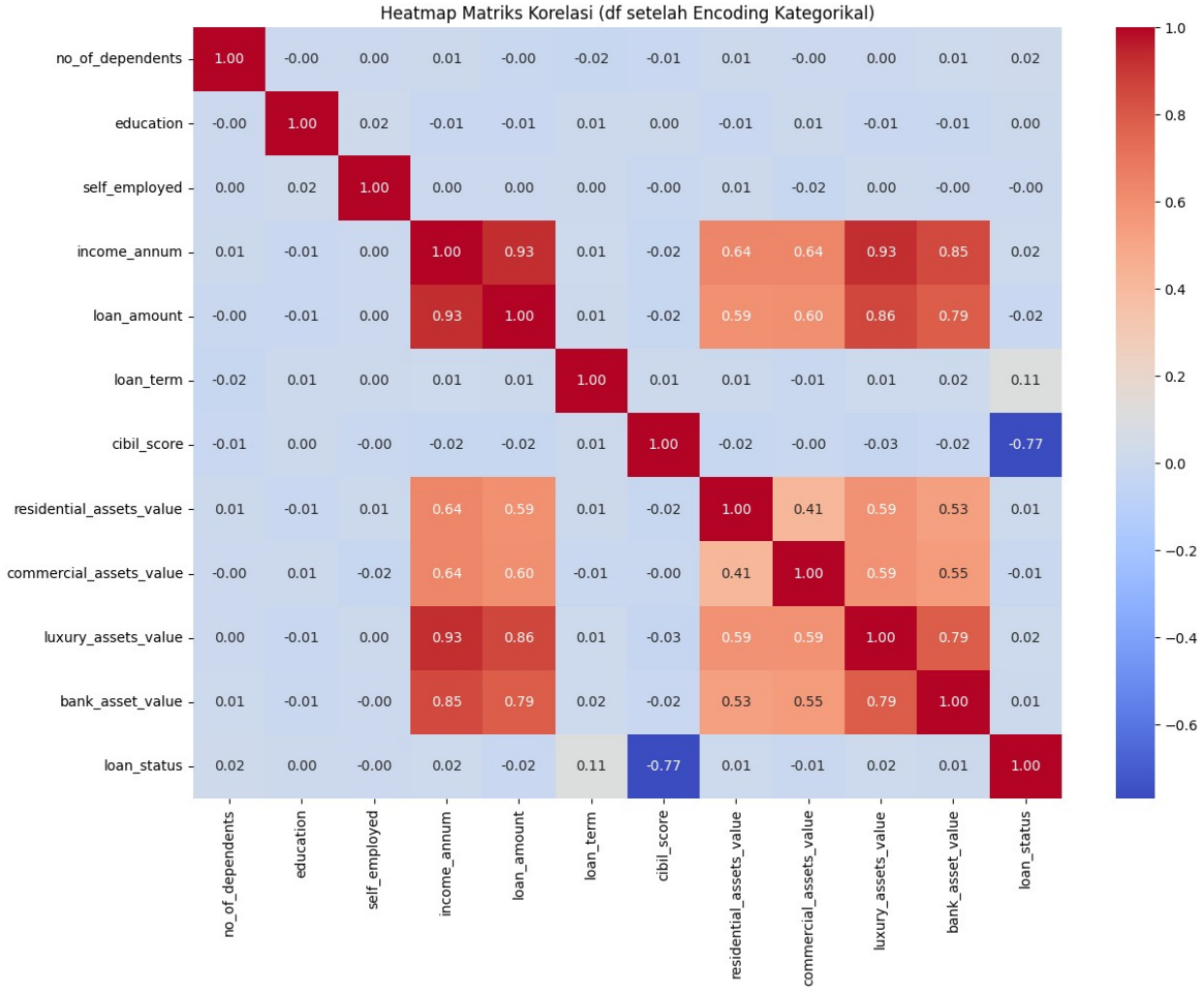
INTERPRETASI

Berdasarkan scatter plot, terlihat bahwa terdapat hubungan positif antara pendapatan tahunan (income_annum) dan jumlah pinjaman (loan_amount). Semakin tinggi pendapatan tahunan yang dimiliki pemohon, semakin besar pula jumlah pinjaman yang diajukan. Hal ini terlihat dari pola titik yang cenderung membentuk tren naik dari kiri bawah ke kanan atas.

Selain itu, berdasarkan warna kategori loan_status, data dengan status Approved dan Rejected tersebar pada berbagai rentang pendapatan maupun jumlah pinjaman. Meskipun demikian, jumlah data dengan status Approved terlihat lebih banyak dibandingkan Rejected pada sebagian besar rentang nilai.

Dalam konteks analisis persetujuan pinjaman, hubungan antara pendapatan tahunan dan jumlah pinjaman penting untuk diperhatikan karena kemampuan finansial pemohon dapat memengaruhi besarnya pinjaman yang diajukan. Variabel income_annum dan loan_amount berpotensi menjadi faktor yang berpengaruh dalam proses klasifikasi status pinjaman.

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

# Membuat salinan df
df_copy = df.copy()

# Membuat LabelEncoder
le = LabelEncoder()

# Mendapatkan kolom kategorikal dari df_copy
categorical_cols_copy =
df_copy.select_dtypes(include='object').columns

# Melakukan encoding pada kolom kategorikal di df_copy
for col in categorical_cols_copy:
    df_copy[col] = le.fit_transform(df_copy[col])

# Menghitung matriks korelasi pada data yang sudah di-encode
correlation_matrix = df_copy.corr()

# Membuat heatmap
plt.figure(figsize=(14, 10))
sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm',
    fmt=".2f"
)
plt.title('Heatmap Matriks Korelasi (df setelah Encoding
Kategorikal)')
plt.show()
```



INTERPRETASI

Berdasarkan heatmap matriks korelasi, sebagian besar variabel memiliki hubungan yang lemah dengan variabel loan_status. Namun, terdapat satu variabel yang menunjukkan hubungan yang cukup kuat, yaitu cibil_score dengan nilai korelasi sebesar -0.77.

Selain itu, terdapat korelasi positif yang kuat antara income_annum dan loan_amount sebesar 0.93. Korelasi yang tinggi juga terlihat antara income_annum dan luxury_assets_value sebesar 0.93, serta antara loan_amount dan luxury_assets_value sebesar 0.86. Hal ini menunjukkan bahwa pemohon dengan pendapatan yang lebih tinggi cenderung memiliki aset yang lebih besar dan mengajukan pinjaman dalam jumlah yang lebih besar.

Dalam konteks analisis persetujuan pinjaman, variabel cibil_score menjadi variabel yang paling berpengaruh terhadap status pinjaman karena memiliki korelasi paling kuat dengan loan_status. Oleh karena itu, cibil_score berpotensi menjadi salah satu variabel penting dalam proses klasifikasi persetujuan pinjaman.

c. Identifikasi Missing Value, Duplikat dan Outlier

Cek Missing Value

```
df.isnull().sum()
```

```
no_of_dependents            0
education                   0
self_employed               0
income_annum                0
loan_amount                 0
loan_term                   0
cibil_score                 0
residential_assets_value    0
commercial_assets_value     0
luxury_assets_value         0
bank_asset_value            0
loan_status                 0
dtype: int64
```

INTERPRETASI

Tidak ada missing value

Cek Data Duplikat

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
# Memeriksa jumlah baris dan kolom dataset
print("Jumlah baris dan kolom:", df.shape)

# Menampilkan jumlah observasi
print("Jumlah data pemohon:", df.shape[0])

# Menampilkan jumlah variabel
print("Jumlah variabel:", df.shape[1])
```

```
Jumlah baris dan kolom: (4269, 12)
Jumlah data pemohon: 4269
Jumlah variabel: 12
```

INTERPRETASI

Berdasarkan hasil pemeriksaan dataset, terdapat 4.269 data pemohon pinjaman (observasi) dan 12 variabel yang digunakan dalam analisis. Jumlah data yang cukup banyak ini dapat memberikan informasi yang representatif untuk proses analisis dan pemodelan.

Selain itu, jumlah variabel yang tersedia mencakup karakteristik pemohon, kondisi keuangan, aset yang dimiliki, serta status pinjaman. Variabel-variabel tersebut dapat digunakan untuk mengidentifikasi faktor-faktor yang memengaruhi persetujuan pinjaman.

Dengan jumlah observasi dan variabel yang memadai, dataset ini dapat digunakan untuk membangun model klasifikasi dalam memprediksi status persetujuan pinjaman.

```
# Memeriksa distribusi variabel target Loan Status
print("\nDistribusi variabel Loan Status:")
print(df["loan_status"].value_counts())

# Menampilkan persentase distribusi Loan Status
print("\nPersentase distribusi Loan Status:")
print(df["loan_status"].value_counts(normalize=True) * 100)
```

```
Distribusi variabel Loan Status:
loan_status
Approved    2656
Rejected    1613
Name: count, dtype: int64

Persentase distribusi Loan Status:
loan_status
Approved    62.215976
Rejected    37.784024
Name: proportion, dtype: float64
```

INTERPRETASI

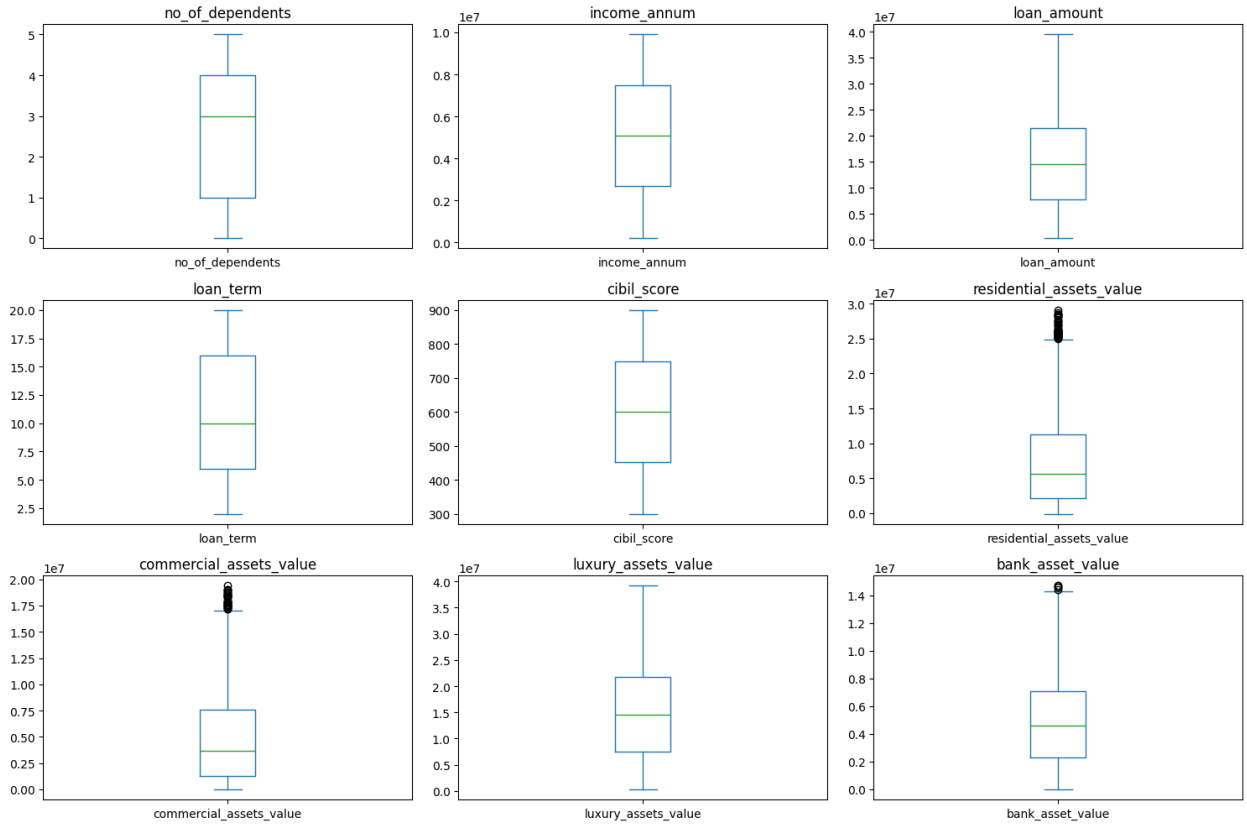
Berdasarkan hasil distribusi variabel target loan_status, terdapat 2.656 data dengan status pinjaman Approved dan 1.613 data dengan status pinjaman Rejected. Hal ini menunjukkan bahwa jumlah pinjaman yang disetujui lebih banyak dibandingkan pinjaman yang ditolak.

Berdasarkan persentase distribusinya, status Approved memiliki proporsi sebesar 62,22%, sedangkan status Rejected sebesar 37,78%. Dengan demikian, mayoritas data pada dataset merupakan pinjaman yang disetujui.

Deteksi Outlier

```
# Menampilkan Boxplot Variabel Numerik

numerical_cols = [
    'no_of_dependents',
    'income_annum',
    'loan_amount',
    'loan_term',
    'cibil_score',
    'residential_assets_value',
    'commercial_assets_value',
    'luxury_assets_value',
    'bank_asset_value'
]

fig, axes = plt.subplots(3, 3, figsize=(15, 10))

for i, col in enumerate(numerical_cols):
```

```
    df[col].plot(kind='box', ax=axes[i // 3, i % 3])
    axes[i // 3, i % 3].set_title(col)

plt.tight_layout()
plt.show()
```



INTERPRETASI

Berdasarkan boxplot seluruh variabel numerik, sebagian besar variabel memiliki sebaran data yang berada dalam rentang yang wajar. Variabel no_of_dependents, income_annum, loan_amount, loan_term, cibil_score, dan luxury_assets_value tidak menunjukkan adanya outlier.

Pada variabel residential_assets_value, commercial_assets_value, dan bank_asset_value terdapat beberapa data yang berada di luar batas whisker boxplot sehingga dapat dikategorikan sebagai outlier. Namun jumlah data tersebut relatif sedikit dibandingkan keseluruhan jumlah observasi dalam dataset.

Dalam konteks analisis persetujuan pinjaman, outlier yang ditemukan masih dapat diterima karena kemungkinan merepresentasikan pemohon yang memiliki nilai aset jauh lebih tinggi dibandingkan pemohon lainnya. Oleh karena itu, data outlier tidak dihapus dan tetap digunakan dalam proses analisis serta pemodelan.

```
# Deteksi Outlier dengan IQR
```

```python
numerical_cols = [
    'no_of_dependents',
    'income_annum',
    'loan_amount',
    'loan_term',
    'cibil_score',
    'residential_assets_value',
    'commercial_assets_value',
    'luxury_assets_value',
    'bank_asset_value'
]

for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[
        (df[col] < lower_bound) |
        (df[col] > upper_bound)
    ]

    print(f"{col}")
    print(f"Jumlah Outlier: {len(outliers)}")
    print("-" * 30)
```

```
no_of_dependents
Jumlah Outlier: 0
------------------------------
income_annum
Jumlah Outlier: 0
------------------------------
loan_amount
Jumlah Outlier: 0
------------------------------
loan_term
Jumlah Outlier: 0
------------------------------
cibil_score
Jumlah Outlier: 0
------------------------------
residential_assets_value
Jumlah Outlier: 52
------------------------------
commercial_assets_value
Jumlah Outlier: 37
```

```
------------------------------
luxury_assets_value
Jumlah Outlier: 0
------------------------------
bank_asset_value
Jumlah Outlier: 5
------------------------------
```

INTERPRETASI

Berdasarkan hasil deteksi outlier menggunakan metode Interquartile Range (IQR), sebagian besar variabel numerik tidak memiliki outlier. Variabel no_of_dependents, income_annum, loan_amount, loan_term, cibil_score, dan luxury_assets_value menunjukkan jumlah outlier sebanyak 0 data.

Namun, terdapat beberapa outlier pada variabel residential_assets_value sebanyak 52 data, commercial_assets_value sebanyak 37 data, dan bank_asset_value sebanyak 5 data. Outlier tersebut menunjukkan adanya beberapa pemohon yang memiliki nilai aset jauh lebih tinggi dibandingkan mayoritas pemohon lainnya.

Dalam konteks analisis persetujuan pinjaman, keberadaan outlier pada variabel aset masih dapat diterima karena kemungkinan merepresentasikan kondisi nyata di mana sebagian pemohon memiliki kekayaan atau aset yang jauh lebih besar dibandingkan pemohon lainnya. Oleh karena itu, outlier tidak dihapus dan tetap dipertahankan untuk proses analisis dan pemodelan selanjutnya.

3. Prapemrosesan dan Rekayasa Fitur

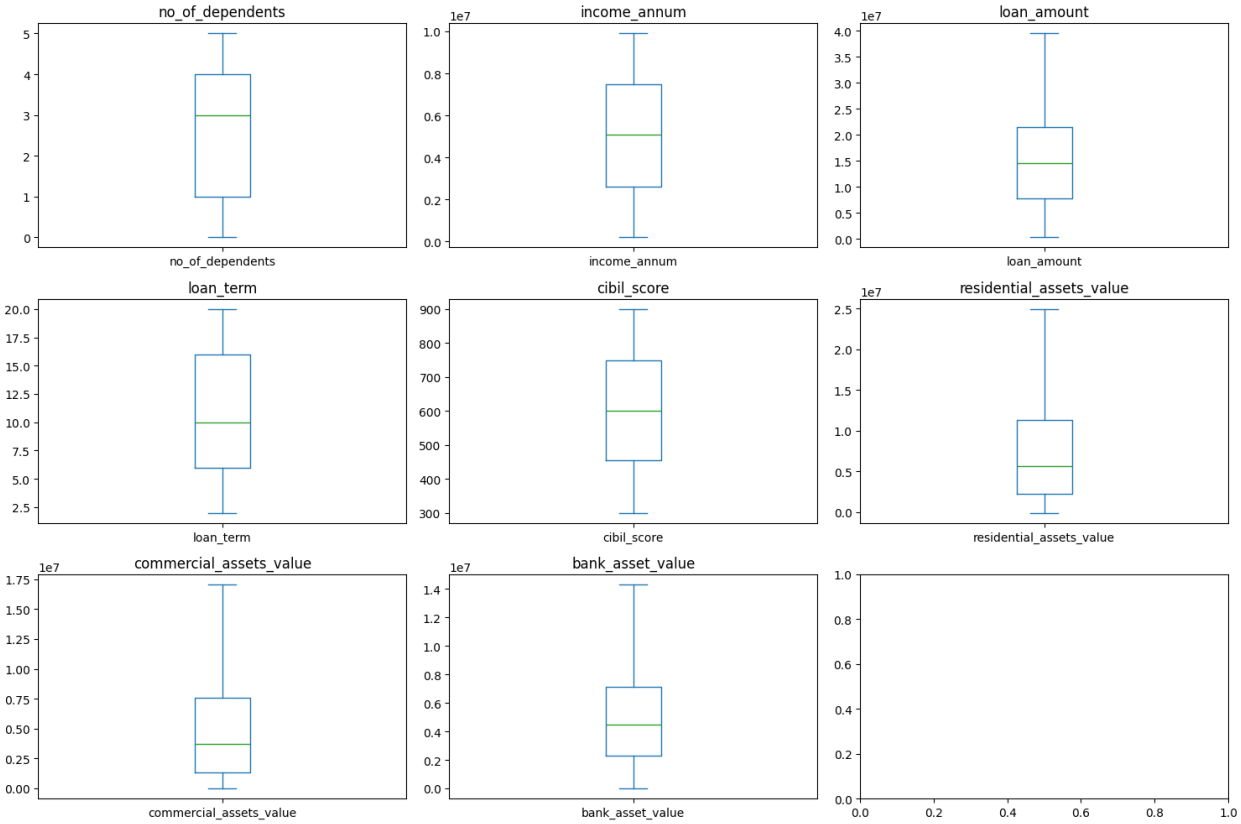
Hapus Variabel yang Memiliki korelasi tinggi

```
# drop variabel Luxury
# Tambahkan baris ini setelah proses pembersihan data selesai atau
sebelum pembagian X dan y
df = df.drop(columns=['luxury_assets_value'])
```

a. Penanganan Outlier

```
# ========================================
# A. DROP BARIS OUTLIER UNTUK BANK_ASSET_VALUE
# ========================================
Q1_bank = df['bank_asset_value'].quantile(0.25)
Q3_bank = df['bank_asset_value'].quantile(0.75)
IQR_bank = Q3_bank - Q1_bank

lower_bank = Q1_bank - 1.5 * IQR_bank
upper_bank = Q3_bank + 1.5 * IQR_bank

# Filter untuk membuang baris yang berada di luar batas (outlier)
```

```
df = df[(df['bank_asset_value'] >= lower_bank) &
(df['bank_asset_value'] <= upper_bank)]


# ========================================
# B. CAPPING OUTLIER RESIDENTIAL & COMMERCIAL
# ========================================
cols_to_cap = ['residential_assets_value', 'commercial_assets_value']

for col in cols_to_cap:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Capping nilai ekstrem ke batas IQR
    df[col] = np.where(df[col] < lower_bound, lower_bound, df[col])
    df[col] = np.where(df[col] > upper_bound, upper_bound, df[col])

# Menampilkan Boxplot Variabel Numerik

numerical_cols = [
    'no_of_dependents',
    'income_annum',
    'loan_amount',
    'loan_term',
    'cibil_score',
    'residential_assets_value',
    'commercial_assets_value',

    'bank_asset_value'
]

fig, axes = plt.subplots(3, 3, figsize=(15, 10))

for i, col in enumerate(numerical_cols):
    df[col].plot(kind='box', ax=axes[i // 3, i % 3])
    axes[i // 3, i % 3].set_title(col)

plt.tight_layout()
plt.show()
```




```
# Deteksi Outlier dengan IQR

numerical_cols = [
    'no_of_dependents',
    'income_annum',
    'loan_amount',
    'loan_term',
    'cibil_score',
    'residential_assets_value',
    'commercial_assets_value',
    'bank_asset_value'
]

for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[
        (df[col] < lower_bound) |
        (df[col] > upper_bound)
```

```
    ]

    print(f"{col}")
    print(f"Jumlah Outlier: {len(outliers)}")
    print("-" * 30)
```

```
no_of_dependents
Jumlah Outlier: 0
------------------------------
income_annum
Jumlah Outlier: 0
------------------------------
loan_amount
Jumlah Outlier: 0
------------------------------
loan_term
Jumlah Outlier: 0
------------------------------
cibil_score
Jumlah Outlier: 0
------------------------------
residential_assets_value
Jumlah Outlier: 0
------------------------------
commercial_assets_value
Jumlah Outlier: 0
------------------------------
bank_asset_value
Jumlah Outlier: 0
------------------------------
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 4264 entries, 0 to 4268
Data columns (total 11 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   no_of_dependents          4264 non-null   int64  
 1   education                 4264 non-null   object 
 2   self_employed             4264 non-null   object 
 3   income_annum              4264 non-null   int64  
 4   loan_amount               4264 non-null   int64  
 5   loan_term                 4264 non-null   int64  
 6   cibil_score               4264 non-null   int64  
 7   residential_assets_value  4264 non-null   float64
 8   commercial_assets_value   4264 non-null   float64
 9   bank_asset_value          4264 non-null   int64  
 10  loan_status               4264 non-null   object 
```

```
dtypes: float64(2), int64(6), object(3)
memory usage: 399.8+ KB
```

```
# Menampilkan jenis data

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 4264 entries, 0 to 4268
Data columns (total 11 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   no_of_dependents          4264 non-null   int64  
 1   education                 4264 non-null   object 
 2   self_employed             4264 non-null   object 
 3   income_annum              4264 non-null   int64  
 4   loan_amount               4264 non-null   int64  
 5   loan_term                 4264 non-null   int64  
 6   cibil_score               4264 non-null   int64  
 7   residential_assets_value  4264 non-null   float64
 8   commercial_assets_value   4264 non-null   float64
 9   bank_asset_value          4264 non-null   int64  
 10  loan_status               4264 non-null   object 
dtypes: float64(2), int64(6), object(3)
memory usage: 399.8+ KB
```

b. Encoding Variabel Numerik

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

categorical_cols = df.select_dtypes(include='object').columns

for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 4264,\n  \"fields\":
[\n    {\n      \"column\": \"no_of_dependents\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1,\n        \"min\": 0,\n        \"max\": 5,\n
\"num_unique_values\": 6,\n        \"samples\": [\n          2,\n
0,\n          1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"education\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0,\n        \"min\": 0,\n
\"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          1,\n          0\n        ],\n        \"semantic_type\":
```

```
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"self_employed\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n
\"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          1,\n          0\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"income_annum\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 2803702,\n        \"min\":
200000,\n        \"max\": 9900000,\n        \"num_unique_values\":
98,\n        \"samples\": [\n          6200000,\n          9300000\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"loan_amount\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
9036876,\n        \"min\": 300000,\n        \"max\": 39500000,\n
\"num_unique_values\": 377,\n        \"samples\": [\n
25100000,\n          31800000\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"loan_term\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 5,\n        \"min\": 2,\n
\"max\": 20,\n        \"num_unique_values\": 10,\n        \"samples\":
[\n          14,\n          8\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"cibil_score\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 172,\n        \"min\": 300,\n
\"max\": 900,\n        \"num_unique_values\": 601,\n
\"samples\": [\n          859,\n          414\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"residential_assets_value\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
6442974.81370635,\n        \"min\": -100000.0,\n        \"max\":
24950000.0,\n        \"num_unique_values\": 251,\n        \"samples\":
[\n          8400000.0,\n          22500000.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"commercial_assets_value\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
4358852.887324881,\n        \"min\": 0.0,\n        \"max\":
17050000.0,\n        \"num_unique_values\": 172,\n        \"samples\":
[\n          2000000.0,\n          14000000.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"bank_asset_value\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
3235326,\n        \"min\": 0,\n        \"max\": 14300000,\n
\"num_unique_values\": 143,\n        \"samples\": [\n
6700000,\n          300000\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"loan_status\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 0,\n
\"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          1,\n          0\n        ],\n        \"semantic_type\":
```

```
\"\",\n        \"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```


c. Data Scaling

Normalisasi

```
from sklearn.preprocessing import MinMaxScaler

# Memilih variabel numerik kontinu
kolom_normalisasi = [
    'income_annum',
    'loan_amount',
    'residential_assets_value',
    'commercial_assets_value',
    'bank_asset_value'
]

# Membuat objek scaler
scaler = MinMaxScaler()

# Menyalin dataframe hasil encoding
df_normalized = df.copy()

# Melakukan normalisasi
df_normalized[kolom_normalisasi] = scaler.fit_transform(
    df_normalized[kolom_normalisasi]
)

# Menampilkan hasil
df_normalized[kolom_normalisasi].head()
```


```
{"summary":"{\n  \"name\": \"df_normalized[kolom_normalisasi]\",\n
\"rows\": 5,\n  \"fields\": [\n    {\n      \"column\":
\"income_annum\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.24249824553175944,\n        \"min\":
0.4020618556701031,\n        \"max\": 0.9896907216494845,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
0.4020618556701031,\n          0.9896907216494845,\n
0.9175257731958762\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"loan_amount\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.19862442137448091,\n        \"min\":
0.30357142857142855,\n        \"max\": 0.7755102040816326,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
0.30357142857142855,\n          0.6096938775510203,\n
0.7499999999999999\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"residential_assets_value\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.2693442490877342,\n
```

```
\"min\": 0.09980039920159679,\n        \"max\": 0.7305389221556886,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
0.11177644710578842,\n          0.499001996007984,\n
0.2874251497005988\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"commercial_assets_value\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.35362483532876887,\n
\"min\": 0.12903225806451613,\n        \"max\": 1.0,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
0.12903225806451613,\n          0.4809384164222874,\n
0.26392961876832843\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"bank_asset_value\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.25295940206553735,\n        \"min\":
0.23076923076923078,\n        \"max\": 0.8951048951048951,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
0.23076923076923078,\n          0.3496503496503497,\n
0.8951048951048951\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


INTERPRETASI

Normalisasi dilakukan menggunakan metode Min-Max Scaling untuk mengubah rentang nilai variabel menjadi antara 0 dan 1. Proses ini bertujuan untuk menyamakan skala antar variabel sehingga fitur dengan nilai besar tidak mendominasi proses analisis maupun pemodelan.

Pada penelitian ini, normalisasi diterapkan pada seluruh variabel numerik kontinu yang berkaitan dengan pendapatan, pinjaman, skor kredit, dan nilai aset.

Standarisasi

```
from sklearn.preprocessing import StandardScaler

# Memilih kolom numerik kontinu
kolom_numerik = [
    'no_of_dependents',
    'cibil_score',
    'loan_term',
]

# Menghitung rata-rata dan standar deviasi
mean = df[kolom_numerik].mean()
std = df[kolom_numerik].std()

# Menerapkan standarisasi
df_standardized = df_normalized.copy()

df_standardized[kolom_numerik] = (
    df_normalized[kolom_numerik] - mean
) / std
```

```
# Menampilkan hasil
df_standardized[kolom_numerik].head()
```


```
{"summary":"{\n  \"name\": \"df_standardized[kolom_numerik]\",\n
\"rows\": 5,\n  \"fields\": [\n    {\n      \"column\":
\"no_of_dependents\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 1.0715474526844664,\n        \"min\": -
1.47356195435494,\n        \"max\": 1.4757753406926868,\n
\"num_unique_values\": 4,\n        \"samples\": [\n          -
1.47356195435494,\n          1.4757753406926868,\n          -
0.29382703633588936\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"cibil_score\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.9110541798898139,\n        \"min\": -
1.2647575941993514,\n        \"max\": 1.0317210654415407,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          -
1.061785995493717,\n          -1.2647575941993514,\n          -
0.5456582159279608\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"loan_term\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 1.063184067877707,\n        \"min\": -
0.507614842442727,\n        \"max\": 1.5955135995094556,\n
\"num_unique_values\": 3,\n        \"samples\": [\n
0.19342797154133384,\n          -0.507614842442727,\n
1.5955135995094556\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


```
df_standardized.head()
```


```
{"summary":"{\n  \"name\": \"df_standardized\",\n  \"rows\": 4264,\n
\"fields\": [\n    {\n      \"column\": \"no_of_dependents\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.9999999999999855,\n        \"min\": -1.47356195435494,\n
\"max\": 1.4757753406926868,\n        \"num_unique_values\": 6,\n
\"samples\": [\n          -0.29382703633588936,\n          -
1.47356195435494,\n          -0.8836944953454148\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"education\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0,\n        \"min\": 0,\n        \"max\": 1,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          1,\n
0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"self_employed\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0,\n        \"min\": 0,\n
\"max\": 1,\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          1,\n          0\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"income_annum\",\n      \"properties\": {\n
```

```
\"dtype\": \"number\",\n        \"std\": 0.289041493772716,\n
\"min\": 0.0,\n        \"max\": 0.9999999999999999,\n
\"num_unique_values\": 98,\n        \"samples\": [\n
0.6185567010309277,\n          0.9381443298969071\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"loan_amount\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.23053256976230502,\n        \"min\": 0.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 377,\n        \"samples\": [\n
0.6326530612244897,\n          0.8035714285714285\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"loan_term\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.9999999999999979,\n        \"min\": -1.5591790634188183,\n
\"max\": 1.5955135995094556,\n        \"num_unique_values\": 10,\n
\"samples\": [\n          0.5439493785333642,\n          -
0.507614842442727\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"cibil_score\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 1.0000000000000018,\n        \"min\": -
1.7402910540239804,\n        \"max\": 1.739222066644038,\n
\"num_unique_values\": 601,\n        \"samples\": [\n
1.5014553367317234,\n          -1.079183561097057\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"residential_assets_value\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.2572045833814917,\n        \"min\": 0.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 251,\n        \"samples\": [\n
0.3393213572854291,\n          0.9021956087824351\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"commercial_assets_value\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.2556511957375302,\n        \"min\": 0.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 172,\n        \"samples\": [\n
0.11730205278592376,\n          0.8211143695014663\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"bank_asset_value\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.22624657712038246,\n        \"min\": 0.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 143,\n        \"samples\": [\n
0.46853146853146854,\n          0.02097902097902098\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"loan_status\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0,\n        \"min\": 0,\n        \"max\": 1,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          1,\n
0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df_standardized"}
```

INTERPRETASI

Standarisasi dilakukan dengan mengubah data sehingga memiliki rata-rata mendekati 0 dan standar deviasi mendekati 1. Proses ini membantu menyamakan distribusi antar variabel dan mengurangi pengaruh perbedaan satuan pengukuran.

Standarisasi diterapkan pada seluruh variabel numerik kontinu agar setiap fitur memiliki skala yang sebanding sehingga dapat meningkatkan performa algoritma machine learning yang sensitif terhadap skala data.

d. Split Data

```
from sklearn.model_selection import train_test_split

# Definisikan variabel target (y) dan fitur (X) menggunakan
df_standardized

# Variabel target (y) adalah kolom 'loan_status'
y = df_standardized['loan_status']

# Variabel prediktor/fitur (X) adalah semua kolom

X = df_standardized.drop(['loan_status'], axis=1)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

4. PEMODELAN DATA

Pembangunan Model

```
# Import library untuk membagi data dan model klasifikasi

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import classification_report, accuracy_score
from sklearn.tree import DecisionTreeClassifier
```

1. Logistik Regression

```python
# --- Logistic Regression ---
print("\nTraining Logistic Regression...")

model_lr = LogisticRegression(
    random_state=42,
    solver='liblinear'
)

model_lr.fit(X_train, y_train)

y_pred_lr = model_lr.predict(X_test)

print("\n--- Logistic Regression Performance ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_lr):.4f}")

print("Classification Report:")
print(classification_report(y_test, y_pred_lr))
```

```
Training Logistic Regression...

--- Logistic Regression Performance ---
Accuracy: 0.9332
Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.96      0.95       531
           1       0.93      0.89      0.91       322

    accuracy                           0.93       853
   macro avg       0.93      0.92      0.93       853
weighted avg       0.93      0.93      0.93       853
```

2. Decision Tree

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# --- Decision Tree ---
print("\nTraining Decision Tree...")

model_dt = DecisionTreeClassifier(
    random_state=42
)

model_dt.fit(X_train, y_train)

y_pred_dt = model_dt.predict(X_test)

print("\n--- Decision Tree Performance ---")
```

```
print(f"Accuracy: {accuracy_score(y_test, y_pred_dt):.4f}")

print("Classification Report:")
print(classification_report(y_test, y_pred_dt))


Training Decision Tree...

--- Decision Tree Performance ---
Accuracy: 0.9859
Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.99      0.99       531
           1       0.99      0.97      0.98       322

    accuracy                           0.99       853
   macro avg       0.99      0.98      0.98       853
weighted avg       0.99      0.99      0.99       853
```

3. Random Forest

```
from sklearn.ensemble import RandomForestClassifier

# --- Random Forest ---
print("\nTraining Random Forest...")

model_rf = RandomForestClassifier(
    random_state=42
)

model_rf.fit(X_train, y_train)

y_pred_rf = model_rf.predict(X_test)

print("\n--- Random Forest Performance ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_rf):.4f}")

print("Classification Report:")
print(classification_report(y_test, y_pred_rf))


Training Random Forest...

--- Random Forest Performance ---
Accuracy: 0.9848
Classification Report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       531
```

```
           1       1.00      0.96      0.98       322

    accuracy                           0.98       853
   macro avg       0.99      0.98      0.98       853
weighted avg       0.99      0.98      0.98       853
```

4. XGBoost

```
from xgboost import XGBClassifier

# --- XGBoost ---
print("\nTraining XGBoost...")

model_xgb = XGBClassifier(
    random_state=42,
    eval_metric='logloss'
)

model_xgb.fit(X_train, y_train)

y_pred_xgb = model_xgb.predict(X_test)

print("\n--- XGBoost Performance ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_xgb):.4f}")

print("Classification Report:")
print(classification_report(y_test, y_pred_xgb))
```

```
Training XGBoost...

--- XGBoost Performance ---
Accuracy: 0.9894
Classification Report:
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       531
           1       1.00      0.97      0.99       322

    accuracy                           0.99       853
   macro avg       0.99      0.99      0.99       853
weighted avg       0.99      0.99      0.99       853
```

5. EVALUASI DATA

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
le.fit(['Approved','Rejected'])

print(dict(zip(le.classes_, le.transform(le.classes_))))
```

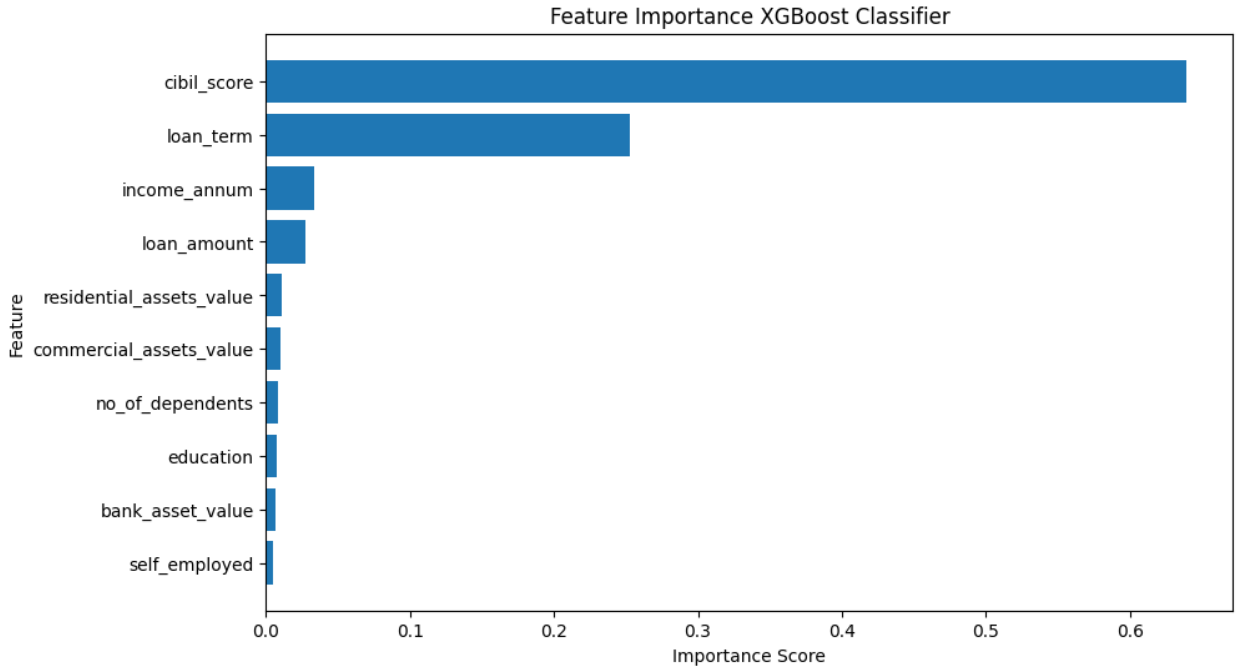
```
{np.str_('Approved'): np.int64(0), np.str_('Rejected'): np.int64(1)}
```

```
from sklearn.metrics import classification_report, accuracy_score

# Evaluate Logistic Regression
y_pred_lr = model_lr.predict(X_test)
accuracy_lr = accuracy_score(y_test, y_pred_lr)

# Evaluate Decision Tree Classifier
y_pred_dt = model_dt.predict(X_test)
accuracy_dt = accuracy_score(y_test, y_pred_dt)

# Evaluate Random Forest Classifier
y_pred_rf = model_rf.predict(X_test)
accuracy_rf = accuracy_score(y_test, y_pred_rf)

# Evaluate XGBoost Classifier
y_pred_xgb = model_xgb.predict(X_test)
accuracy_xgb = accuracy_score(y_test, y_pred_xgb)

model_performance = {
    'Logistic Regression': accuracy_lr,
    'Decision Tree Classifier': accuracy_dt,
    'Random Forest Classifier': accuracy_rf,
    'XGBoost Classifier': accuracy_xgb
}

best_model_name = max(
    model_performance,
    key=model_performance.get
)

best_accuracy = model_performance[best_model_name]

print("\n### Ringkasan Kinerja Model:\n")

for model, accuracy in model_performance.items():
    print(f"* {model}: Accuracy = {accuracy:.4f}")

print(
    f"\nBerdasarkan hasil ini, {best_model_name} "
    f"adalah model terbaik dengan akurasi "
    f"{best_accuracy:.4f} pada data uji.\n"
)

print(
```

```
    f"\n--- Classification Report untuk "
    f"{best_model_name} ---"
)

if best_model_name == 'Logistic Regression':
    print(classification_report(y_test, y_pred_lr))

elif best_model_name == 'Decision Tree Classifier':
    print(classification_report(y_test, y_pred_dt))

elif best_model_name == 'Random Forest Classifier':
    print(classification_report(y_test, y_pred_rf))

elif best_model_name == 'XGBoost Classifier':
    print(classification_report(y_test, y_pred_xgb))


### Ringkasan Kinerja Model:

* Logistic Regression: Accuracy = 0.9332
* Decision Tree Classifier: Accuracy = 0.9859
* Random Forest Classifier: Accuracy = 0.9848
* XGBoost Classifier: Accuracy = 0.9894

Berdasarkan hasil ini, XGBoost Classifier adalah model terbaik dengan
akurasi 0.9894 pada data uji.


--- Classification Report untuk XGBoost Classifier ---
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       531
           1       1.00      0.97      0.99       322

    accuracy                           0.99       853
   macro avg       0.99      0.99      0.99       853
weighted avg       0.99      0.99      0.99       853


import pandas as pd
import matplotlib.pyplot as plt

feature_importance = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': model_xgb.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by='Importance',
    ascending=False
)
```

```
print(feature_importance)

plt.figure(figsize=(10,6))

plt.barh(
    feature_importance['Feature'],
    feature_importance['Importance']
)

plt.title('Feature Importance XGBoost Classifier')
plt.xlabel('Importance Score')
plt.ylabel('Feature')

plt.gca().invert_yaxis()

plt.show()
```

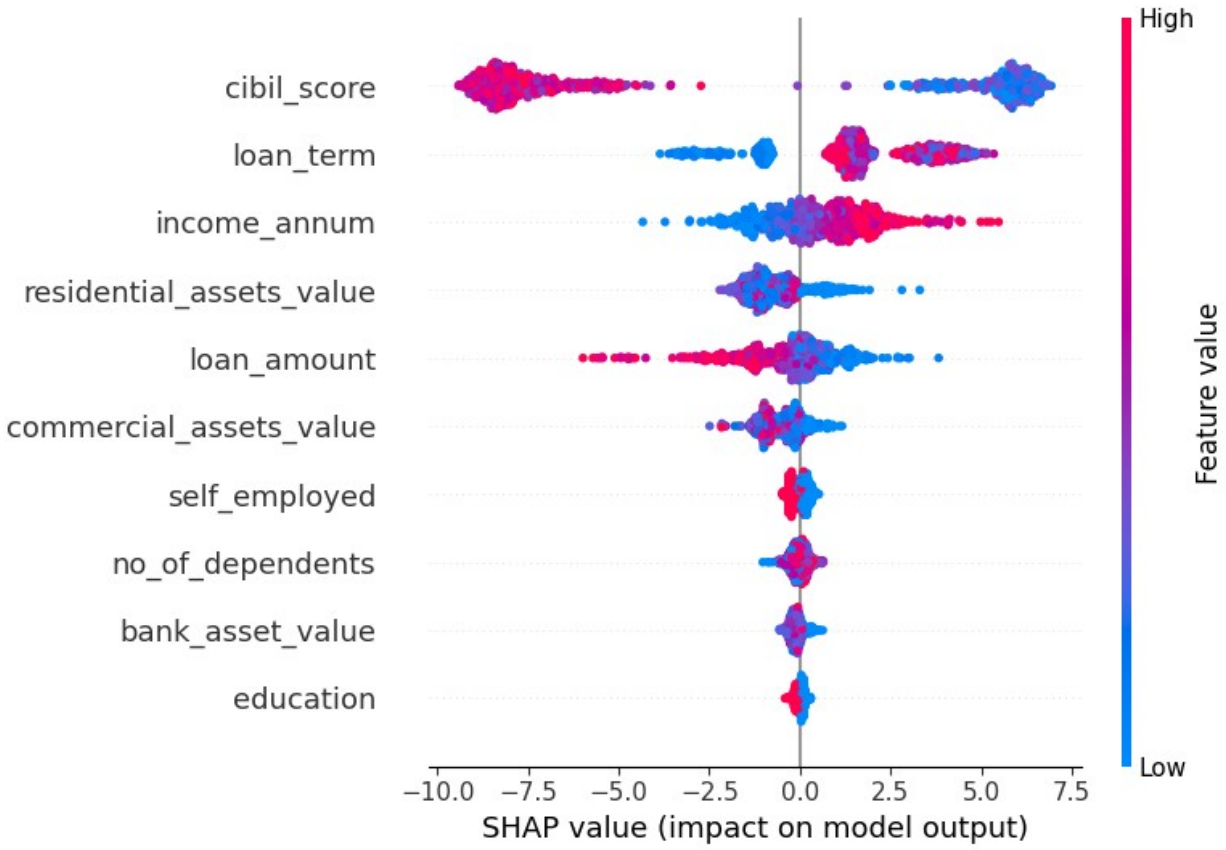
	Feature	Importance
6	cibil_score	0.639698
5	loan_term	0.252494
3	income_annum	0.033080
4	loan_amount	0.027235
7	residential_assets_value	0.010899
8	commercial_assets_value	0.010437
0	no_of_dependents	0.007904
1	education	0.007078
9	bank_asset_value	0.006411
2	self_employed	0.004764



```
# Install SHAP (jalankan sekali saja)
!pip install shap

import shap

# 'model_xgb' adalah model terbaik yang sudah dilatih sebelumnya

# Menghitung Skor SHAP
explainer = shap.TreeExplainer(model_xgb)

# shap_values untuk klasifikasi biner seringkali menghasilkan array 2D
secara langsung (n_samples, n_features)
# atau daftar dua array [shap_values_class_0, shap_values_class_1].
# Berdasarkan error, 'shap_values' sepertinya sudah merupakan array 2D
untuk kelas positif.
shap_values = explainer.shap_values(X_test)

# Menampilkan SHAP Summary Plot
# Jika shap_values adalah array 2D (n_samples, n_features), gunakan
langsung.
# Jika shap_values adalah list dari array (untuk setiap kelas),
gunakan shap_values[1] untuk kelas positif (jika ada).
# Berdasarkan konteks error, `shap_values` sudah merupakan matrix yang
dibutuhkan.
shap.summary_plot(shap_values, X_test, feature_names=X_test.columns)
```

```
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-
packages (0.52.0)
Requirement already satisfied: numpy>=2 in
/usr/local/lib/python3.12/dist-packages (from shap) (2.0.2)
Requirement already satisfied: scipy in
/usr/local/lib/python3.12/dist-packages (from shap) (1.16.3)
Requirement already satisfied: scikit-learn in
/usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in
/usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in
/usr/local/lib/python3.12/dist-packages (from shap) (4.67.3)
Requirement already satisfied: packaging>20.9 in
/usr/local/lib/python3.12/dist-packages (from shap) (26.2)
Requirement already satisfied: slicer==0.0.8 in
/usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba in
/usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: llvmlite in
/usr/local/lib/python3.12/dist-packages (from shap) (0.43.0)
Requirement already satisfied: cloudpickle in
/usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas->shap)
```

```
(2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.12/dist-packages (from pandas->shap) (2026.2)
Requirement already satisfied: joblib>=1.2.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn->shap)
(1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn->shap)
(3.6.0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2-
>pandas->shap) (1.17.0)
```



Hyperparameter Tuning

```python
# Import GridSearchCV untuk hyperparameter tuning
from sklearn.model_selection import GridSearchCV
from xgboost import XGBClassifier

# Definisikan grid parameter untuk XGBoost
param_grid_xgb = {
```

```
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.05, 0.1],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}

print("Memulai GridSearchCV untuk XGBoost...")

# Inisialisasi GridSearchCV
grid_search_xgb = GridSearchCV(
    estimator=XGBClassifier(
        random_state=42,
        eval_metric='logloss'
    ),
    param_grid=param_grid_xgb,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

# Fit GridSearchCV ke data training
grid_search_xgb.fit(X_train, y_train)

print("GridSearchCV selesai.")

# Parameter terbaik
best_params_xgb = grid_search_xgb.best_params_
best_score_xgb = grid_search_xgb.best_score_

print(f"\nParameter terbaik untuk XGBoost: {best_params_xgb}")
print(f"Akurasi cross-validation terbaik untuk XGBoost:
{best_score_xgb:.4f}")

# Model terbaik
best_model_xgb = grid_search_xgb.best_estimator_

# Prediksi data test
y_pred_xgb_tuned = best_model_xgb.predict(X_test)

print("\n--- Kinerja Tuned XGBoost Classifier pada Test Set ---")

print(f"Accuracy: {accuracy_score(y_test, y_pred_xgb_tuned):.4f}")

print("Classification Report:")
print(classification_report(y_test, y_pred_xgb_tuned))
```

```
Memulai GridSearchCV untuk XGBoost...
Fitting 5 folds for each of 108 candidates, totalling 540 fits
GridSearchCV selesai.
```

```
Parameter terbaik untuk XGBoost: {'colsample_bytree': 0.8,
'learning_rate': 0.1, 'max_depth': 5, 'n_estimators': 100,
'subsample': 1.0}
Akurasi cross-validation terbaik untuk XGBoost: 0.9850

--- Kinerja Tuned XGBoost Classifier pada Test Set ---
Accuracy: 0.9906
Classification Report:
              precision    recall  f1-score   support

           0       0.99      1.00      0.99       531
           1       1.00      0.98      0.99       322

    accuracy                           0.99       853
   macro avg       0.99      0.99      0.99       853
weighted avg       0.99      0.99      0.99       853
```

Model Final

```python
from sklearn.metrics import accuracy_score, recall_score,
classification_report

# ================================================
# KINERJA MODEL XGBOOST AWAL
# ================================================

report_initial_xgb = classification_report(
    y_test,
    y_pred_xgb,
    output_dict=True
)

recall_initial_xgb = report_initial_xgb['1']['recall']

print("=== Kinerja XGBoost (Awal) ===")

print(f"Accuracy: {accuracy_xgb:.4f}")
print(f"Recall (Rejected Class): {recall_initial_xgb:.4f}")

# ================================================
# KINERJA MODEL XGBOOST TUNED
# ================================================

accuracy_tuned_xgb = accuracy_score(
    y_test,
    y_pred_xgb_tuned
)
```

```python
report_tuned_xgb = classification_report(
    y_test,
    y_pred_xgb_tuned,
    output_dict=True
)

recall_tuned_xgb = report_tuned_xgb['1']['recall']

print("\n=== Kinerja XGBoost (Tuned) ===")

print(f"Accuracy: {accuracy_tuned_xgb:.4f}")
print(f"Recall (Rejected Class): {recall_tuned_xgb:.4f}")

# ================================================
# PERBANDINGAN
# ================================================

print("\n--- Perbandingan ---")

print(
    f"Peningkatan Akurasi: "
    f"{(accuracy_tuned_xgb - accuracy_xgb):.4f}"
)

print(
    f"Peningkatan Recall (Rejected): "
    f"{(recall_tuned_xgb - recall_initial_xgb):.4f}"
)
```

```
=== Kinerja XGBoost (Awal) ===
Accuracy: 0.9894
Recall (Rejected Class): 0.9720

=== Kinerja XGBoost (Tuned) ===
Accuracy: 0.9906
Recall (Rejected Class): 0.9783

--- Perbandingan ---
Peningkatan Akurasi: 0.0012
Peningkatan Recall (Rejected): 0.0062
```

```python
# ====================================
# ANALISIS OVERFITTING / UNDERFITTING
# XGBOOST TUNED
# ====================================

from sklearn.metrics import accuracy_score

# Prediksi data training
y_train_pred = best_model_xgb.predict(X_train)
```

```python
# Prediksi data testing
y_test_pred = best_model_xgb.predict(X_test)

# Accuracy training dan testing
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print("=== ANALISIS OVERFITTING / UNDERFITTING ===")

print(f"Training Accuracy : {train_accuracy:.4f}")
print(f"Testing Accuracy  : {test_accuracy:.4f}")

gap = abs(train_accuracy - test_accuracy)

print(f"Selisih Accuracy  : {gap:.4f}")

if gap > 0.05:
    print("\nKesimpulan: Model cenderung mengalami OVERFITTING")
elif train_accuracy < 0.80 and test_accuracy < 0.80:
    print("\nKesimpulan: Model cenderung mengalami UNDERFITTING")
else:
    print("\nKesimpulan: Model memiliki GENERALISASI yang baik")
```

```
=== ANALISIS OVERFITTING / UNDERFITTING ===
Training Accuracy : 0.9982
Testing Accuracy  : 0.9906
Selisih Accuracy  : 0.0076

Kesimpulan: Model memiliki GENERALISASI yang baik
```

```python
from sklearn.model_selection import learning_curve
import numpy as np
import matplotlib.pyplot as plt

def plot_learning_curve(estimator, title, X, y, axes=None, ylim=None,
cv=None,
                        n_jobs=None, train_sizes=np.linspace(.1, 1.0,
5)):
    """
    Generate 3 plots: the test and training learning curve, the
training
    samples vs fit times curve, the fit times vs score curve.

    Parameters
    ----------
    estimator : estimator instance
        An estimator instance implementing the "fit" and "predict"
methods
        which will be cloned for each validation.

    title : string
```

```
        Title for the chart.

    X : array-like, shape (n_samples, n_features)
        Training vector, where n_samples is the number of samples and
        n_features is the number of features.

    y : array-like, shape (n_samples) or (n_samples, n_features),
        optional
        Target relative to X for classification or regression; None
for
        unsupervised learning.

    axes : array-like, shape (3,), optional
        Axes to plot the curves on.

    ylim : tuple, shape (2,), optional
        Defines limits for the y-axis.

    cv : int, cross-validation generator or an iterable, optional
        Determines the cross-validation splitting strategy.
        Possible inputs for cv are:
          - None, to use the default 5-fold cross-validation,
          - integer, to specify the number of folds.
          - :term:`CV splitter`,
          - An iterable yielding (train, test) splits as arrays of
indices.

        For integer/None inputs, if ``y`` is binary or multiclass,
        :class:`StratifiedKFold` used. If the estimator is not a
classifier
        or if ``y`` is neither binary nor multiclass, :class:`KFold`
is used.

        Refer :ref:`User Guide <cross_validation>` for the various
        cross-validation strategies that can be used here.

    n_jobs : int or None, optional (default=None)
        Number of jobs to run in parallel.
        ``-1`` means using all processors.

    train_sizes : array-like, shape (n_ticks,),
        defaults to np.linspace(0.1, 1.0, 5)
        Relative or absolute numbers of training examples that will be
used to
        generate the learning curve.
    """
    if axes is None:
        _, axes = plt.subplots(1, 3, figsize=(20, 5))

    axes[0].set_title(title)
```

```
    if ylim is not None:
        axes[0].set_ylim(*ylim)
    axes[0].set_xlabel("Training examples")
    axes[0].set_ylabel("Score")

    train_sizes, train_scores, test_scores, fit_times, _ = \
        learning_curve(estimator, X, y, cv=cv, n_jobs=n_jobs,
                       train_sizes=train_sizes,
                       return_times=True)
    train_scores_mean = np.mean(train_scores, axis=1)
    train_scores_std = np.std(train_scores, axis=1)
    test_scores_mean = np.mean(test_scores, axis=1)
    test_scores_std = np.std(test_scores, axis=1)
    fit_times_mean = np.mean(fit_times, axis=1)
    fit_times_std = np.std(fit_times, axis=1)

    # Plot learning curve
    axes[0].grid()
    axes[0].fill_between(train_sizes, train_scores_mean -
train_scores_std,
                         train_scores_mean + train_scores_std,
alpha=0.1,
                         color="r")
    axes[0].fill_between(train_sizes, test_scores_mean -
test_scores_std,
                         test_scores_mean + test_scores_std,
alpha=0.1,
                         color="g")
    axes[0].plot(train_sizes, train_scores_mean, 'o-', color="r",
                 label="Training score")
    axes[0].plot(train_sizes, test_scores_mean, 'o-', color="g",
                 label="Cross-validation score")
    axes[0].legend(loc="best")

    # Plot n_samples vs fit_times
    axes[1].grid()
    axes[1].plot(train_sizes, fit_times_mean, 'o-')
    axes[1].fill_between(train_sizes, fit_times_mean - fit_times_std,
                         fit_times_mean + fit_times_std, alpha=0.1)
    axes[1].set_xlabel("Training examples")
    axes[1].set_ylabel("fit_times")
    axes[1].set_title("Scalability of the model")

    # Plot fit_time vs score
    axes[2].grid()
    axes[2].plot(fit_times_mean, test_scores_mean, 'o-')
    axes[2].fill_between(fit_times_mean, test_scores_mean -
test_scores_std,
                         fit_times_mean + test_scores_std, alpha=0.1)
    axes[2].set_xlabel("fit_times")
```

```
    axes[2].set_ylabel("Score")
    axes[2].set_title("Performance of the model")

    return plt


# ====================================
# LEARNING CURVE XGBOOST TUNED
# ====================================

fig, axes = plt.subplots(1, 3, figsize=(20, 5))

plot_learning_curve(
    estimator=best_model_xgb,
    title="Learning Curves (XGBoost Tuned)",
    X=X_train,
    y=y_train,
    axes=axes,
    ylim=(0.7, 1.01),
    cv=5,
    n_jobs=-1
)

plt.tight_layout()
plt.show()
```